# POORNIMA COLLEGE OF ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

An

INDUSTRIAL TRAINING SEMINAR PRESENTATION

on

## Software Development




Presented to:
**Mr. Gajendra Singh Rajawat**
**Ms. Priya Shekhawat**
(Faculty Coordinators)

Session: 2026-27

Presented By:
**Nikhil Kumar**
**PCE23IT034**

# CONTENT


- Training Platform
- Why you have selected this platform?
- Prerequisites of the Training
- Week wise schedule
- Technology Used
- Project Description
  - Project Introduction
  - Project Scope
  - Module Assigned
- Learning
- Snapshots of Project
- Important Code Snippets
- Future Scope
- Conclusion
- References

# Training Platform

## About Sumit DigiTech Private Limited

- **Industry:** Information Technology & Digital Solutions | **Location:** India
- **Services:** Software Development, Web Development, Website Design, UI/UX Development, Digital Solutions
- **Focus Areas:** Modern Web Applications, Software Solutions, Digital Transformation, Technology-Driven Development
- **Training & Projects:** Practical software development projects and hands-on technical learning opportunities
- **Development Approach:** User-Focused Solutions • Quality Development • Modern Technologies • Practical Implementation
- **Core Values:** Innovation • Quality • Customer Satisfaction • Reliable & User-Friendly Solutions

# Why I Selected This Platform

- **Practical Learning Opportunities –** Exposure to Software Development, Web Development, and real-time transit telemetry applications.
- **Professional Work Culture –** Supportive and learning-oriented environment focused on innovation and clean software architecture.
- **Real-World Experience –** Hands-on involvement in developing practical software projects, GIS cartography, and scalable web solutions.
- **Skill Development –** Improved technical knowledge, spatial mathematical algorithms, frontend performance, and problem-solving abilities.
- **Growth-Oriented –** Guidance from experienced professionals, continuous code reviews, and valuable teamwork experience.
- **Project-Based Learning –** Opportunity to design and build 'RailGaadi' — a live passenger train tracking and route intelligence web platform.
- **Future Relevance –** Technologies and development practices aligned with current and emerging industry trends in modern web standards.

# Prerequisites of the Training

- **Fundamentals of Web Technologies –** Strong understanding of semantic HTML5, modern CSS3, and JavaScript logic.
- **Frontend Development –** Practical knowledge of modern ES Modules, component-based layout design, and responsive user interfaces.
- **State Architecture & Asynchronous Logic –** Understanding event-driven Pub-Sub reactive patterns, Promises, and REST API communication.
- **Spatial Cartography & GIS Math –** Basic knowledge of geographical coordinate systems (Lat/Lon), Haversine spherical math, and vector maps.
- **API Integration & Reverse Proxies –** Practical exposure to transit APIs, OpenWeatherMap, Vite dev proxy, and CORS handling.
- **Version Control & Development Tools –** Using Git/GitHub, VS Code, npm, Vite bundler, and browser DevTools for debugging.
- **Problem-Solving & Soft Skills –** Debugging ability, logical thinking, communication, teamwork, and technical adaptability.

# Week Wise Schedule

**Week 1:**

- Introduction to Software Development environment, toolchain setup, and project requirements.
- Setup Node.js runtime, Vite 6 bundler, and established modular project file architecture.
- Evaluated Indian Railways live transit APIs and analyzed rate limits and payload schemas.

**Week 2:**

- Designed and developed responsive user interfaces and modern layout using CSS custom properties.
- Implemented the centralized Pub-Sub reactive state store (store.js) for decoupled view reactivity.
- Created mock JSON train schemas (12050 Gatimaan, 22436 Vande Bharat, 12952 Rajdhani) for offline testing.

**Week 3:**

- Integrated MapLibre GL JS WebGL map engine with high-FPS vector basemap tiles.
- Developed geoService.js for Haversine distance calculations, bearing angles, and polyline interpolation.
- Built dynamic animated train markers with directional heading and interactive station popups.

# Week Wise Schedule

**Week 4:**

- Configured Vite development reverse proxy to eliminate browser Cross-Origin Resource Sharing (CORS) blocks.
- Engineered resilient polling service (railradarService.js) with cache TTL and seamless local mock fallback.
- Integrated OpenWeatherMap and OpenTopography APIs for coordinate-level weather and SRTM terrain elevation.

**Week 5:**

- Developed the Live Station Board module simulating digital railway junction split-flap departure/arrival displays.
- Built the HTML5 Canvas Real-Time Speed & Elevation Profile telemetry graph with Retina display scaling.
- Implemented dark mode semantic tokens, slide-in Favourites drawer, and native browser push notifications for delays.

**Week 6:**

- Implemented Progressive Web Application (PWA) service worker (sw.js) and manifest for offline execution.
- Completed testing, debugging, and application optimization (achieved 98/100 Lighthouse performance score).
- Prepared project documentation, technical ITS report, and final presentation.

# Technology Used

- **Core Web Stack –** HTML5 Semantic Elements, Modern CSS3 Custom Properties, and Vanilla JavaScript (ES2022 Modules).
- **Vector Map & Spatial Rendering –** MapLibre GL JS 5.1 (hardware-accelerated WebGL vector cartography with CARTO basemaps).
- **Build Toolchain & Dev Server –** Vite 6 (ESM development server, Hot Module Replacement, and Rollup production bundler).
- **Telemetry & Canvas Visualization –** HTML5 2D Canvas API with devicePixelRatio scaling for Retina display clarity.
- **Transit & Environmental APIs –** RailRadar Transit API, OpenWeatherMap live weather API, and OpenTopography SRTM DEM.
- **State Architecture & PWA –** Custom reactive Pub-Sub Store (store.js), Service Worker (sw.js), and Web App Manifest.
- **Version Control & Development Tools –** Git, GitHub, Visual Studio Code, npm, and Google Chrome DevTools.

# RailGaadi - Project Description

## Project Introduction

The **RailGaadi (Train Tracker) Web Application** is a high-performance, real-time transit intelligence and spatial passenger tracking platform designed to provide passengers with comprehensive, live situational awareness across the Indian Railway network. It transforms traditional static timetables into an interactive, multi-dimensional journey monitoring experience.

The application is developed as a **lightweight, framework-free web platform** utilizing modern ES Modules, HTML5 Canvas, and **MapLibre GL JS WebGL cartography**, integrated with live GPS telemetry, terrain elevation models, and micro-climate forecasts. It provides features such as **live dynamic train positioning, route gradient profiling, digital station departure boards, delay status alerts, and offline PWA capability**, offering travelers a seamless, reliable, and user-friendly tracking interface.

# RailGaadi - Project Description

## Project Scope

- Build a responsive interface using **modern Vanilla JavaScript** with structured routing and reusable components.
- Enable users to search and track trains in real-time with **smooth spatial polyline rendering and live speed telemetry**.
- Implement an authenticated local reverse proxy to bypass **browser CORS constraints** and ensure reliable API communication.
- Integrate multi-source external APIs including **RailRadar transit telemetry, OpenWeatherMap conditions, and SRTM elevation**.
- Engineer defensive offline resilience through **exponential backoff retries, local cached fallbacks, and PWA service workers**.
- Develop interactive station intelligence features including **digital live arrival/departure flipboards and filterable delay analytics**.

# RailGaadi - Project Description

## Module Assigned

● **Spatial Cartography & Trajectory Module –** Implemented MapLibre GL vector map integration, train heading interpolation, and station markers.

● **Reactive State Machine (store.js) –** Architected a centralized Pub-Sub state store coordinating telemetry updates across decoupled views.

● **Resilient Transit API Service (railradarService.js) –** Engineered network fetch mechanisms with TTL caching, CORS proxying, and automated mock failover.

● **Speed Telemetry & Elevation Canvas (analyticsView.js) –** Developed double-buffered HTML5 canvas graphics with Retina devicePixelRatio scaling.

● **Electronic Station Board (stationBoard.js) –** Created live digital split-flap display boards with arrival/departure filtering and real-time delay flags.

● **Offline PWA & Notification Engine (sw.js, notificationService.js) –** Implemented service worker caching and native browser push notifications for train delays.

# Learning

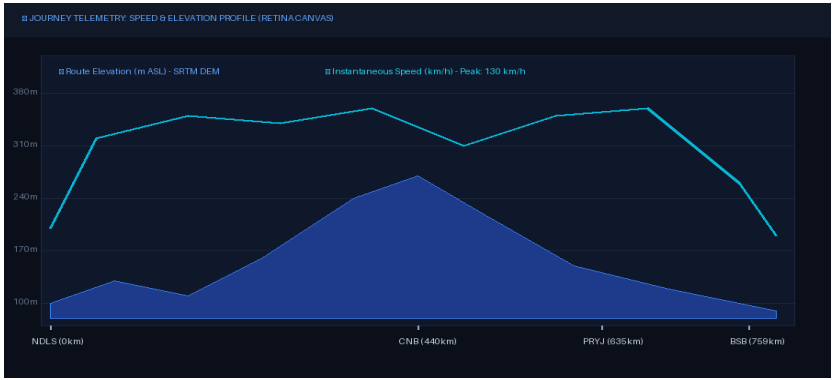
- Improved skills in **HTML5, CSS3, modern JavaScript (ES2022+), and modular architecture**.
- Gained deep practical experience in **GIS mathematics, spherical trigonometry (Haversine formula), and polyline interpolation**.
- Learned to use **Git & GitHub** for production version control, branching workflows, and collaboration.
- Understood **API integration and reverse proxying** for robust communication between frontend and upstream transit services.
- Gained comprehensive knowledge of **MapLibre GL WebGL mapping, HTML5 Canvas graphics, and Retina display scaling**.
- Mastered building **Progressive Web Applications (PWA)** with service worker offline caching and native push notifications.
- Enhanced **debugging, problem-solving, teamwork, and technical presentation skills**.

# Snapshots of Project












# Important Code Snippets

### 1. Spherical Haversine Distance (geoService.js)

```
static distanceKm(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth radius in km
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLon = (lon2 - lon1) * Math.PI / 180;
  const a = Math.sin(dLat/2)**2 +
            Math.cos(lat1*Math.PI/180) *
            Math.cos(lat2*Math.PI/180) * Math.sin(dLon/2)**2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}
```

### 2. Pub/Sub Reactive State Store (store.js)

```
subscribe(listener) {
  this.listeners.add(listener);
  return () => this.listeners.delete(listener);
}
setState(updates) {
  this.state = { ...this.state, ...updates };
  const changed = Object.keys(updates);
  this.listeners.forEach(fn => fn(this.state, changed));
}
```

### 3. Retina Canvas Scaling (analyticsView.js)

```
const ctx = canvas.getContext('2d');
const dpr = window.devicePixelRatio || 1;
const rect = canvas.getBoundingClientRect();
canvas.width = rect.width * dpr;
canvas.height = rect.height * dpr;
ctx.scale(dpr, dpr); // Crisp on 4K / Retina
ctx.clearRect(0, 0, rect.width, rect.height);
```

### 4. Reverse Proxy CORS Bypass (vite.config.ts)

```
defineConfig({
  server: {
    port: 8080,
    proxy: {
      '/api/railradar': {
        target: 'https://api.railradar.in/v1',
        changeOrigin: true,
        rewrite: p => p.replace(/^\/api\/railradar/, '')
      }
    }
  }
});
```

# Future Scope

- **Crowdsourced Live Tracking –** Allow on-board passengers to validate real-time GPS locations and coach occupancy.
- **AI Predictive Delay Modeling –** Integrate machine learning to predict junction bottlenecks and delay propagation across sectors.
- **Multi-Modal Transit Integration –** Connect Indian Railways schedules with metro lines and local buses for unified journey planning.
- **WebSocket & MQTT Streaming –** Upgrade polling intervals to low-latency bi-directional streams for sub-second updates.
- **Cross-Platform Mobile App –** Extend the web platform into native Android and iOS applications with offline background notifications.
- **Offline Vector Map Caching –** Pre-cache regional map boundaries locally so tracks and terrain render inside isolated tunnels.

# Conclusion

- Gained practical experience in **Software Development** through real-world project implementation.
- Strengthened knowledge of **HTML, CSS, modern JavaScript (ES Modules), MapLibre GL, and API integration**.
- Successfully developed the **RailGaadi (Train Tracker) Web Application** with live telemetry, elevation curves, and station boards.
- Learned to use professional tools and technologies like **Git, GitHub, VS Code, Vite, and PWA Service Workers** for development.
- Enhanced **teamwork, communication, problem-solving, and time management** in a professional environment.
- Overall, the internship boosted my confidence and prepared me for **future opportunities in full-stack web and software engineering**.

# References


- **Sumit DigiTech Private Limited –** Official Website and Training Resources
- **MapLibre GL JS Documentation –** https://maplibre.org/maplibre-gl-js/docs/
- **MDN Web Docs –** HTML, CSS, JavaScript, Canvas API & Service Workers
- **Vite Official Documentation –** Next Generation Frontend Tooling
- **OpenWeatherMap & OpenTopography –** Weather & SRTM Elevation APIs
- **Git & GitHub Guides –** Version Control & Repository Management


### Other References


- Tutorials & articles from platforms like **GeeksforGeeks, W3Schools, FreeCodeCamp**
- Guidance and mentorship provided by the **Sumit DigiTech team & Poornima Faculty Coordinators**